

CS224N Lecture 9: Pre-training (Part 2)

Stanford University

Week 5

Abstract

This lecture continues the exploration of pre-training in NLP, focusing on decoder-only models and the GPT family. We cover parameter-efficient fine-tuning methods, encoder-decoder pre-training with span corruption, the evolution from GPT-1 to GPT-3, in-context learning, scaling laws, and chain-of-thought prompting. The lecture concludes with a discussion of what these models learn and their emergent capabilities.

Contents

1	Parameter-Efficient Fine-tuning	3
1.1	Motivation	3
1.2	Full Fine-tuning vs Parameter-Efficient Fine-tuning	3
1.3	Prefix Tuning / Prompt Tuning	3
1.4	LoRA: Low-Rank Adaptation	4
1.5	Comparison of Fine-tuning Methods	5
2	Encoder-Decoder Pre-training	5
2.1	Challenge: Combining Bidirectional and Generative	5
2.2	Span Corruption (T5)	6
2.3	T5: Text-to-Text Transfer Transformer	6
2.4	Implicit Knowledge Retrieval	7
3	Decoder-Only Models: The GPT Family	7
3.1	Why Decoder-Only Models?	7
3.2	GPT-1 (2018)	7
3.2.1	Fine-tuning for Tasks	7
3.3	GPT-2 (2019): Scaling Up	8
3.4	GPT-3 (2020): Emergence of In-Context Learning	8
3.5	In-Context Learning	8
3.5.1	Types of In-Context Learning	9
3.6	More In-Context Learning Examples	9
4	Scaling Laws and Compute-Optimal Training	10
4.1	The Scaling Hypothesis	10
4.2	The Compute Budget Problem	10
4.3	Chinchilla Scaling Laws	10
4.4	Scaling Law Visualization	11

5	Chain-of-Thought Prompting	11
5.1	Motivation	11
5.2	Chain-of-Thought Prompting	12
5.3	Why Does Chain-of-Thought Work?	12
5.4	Performance Gains	13
6	What Do Pre-trained Models Learn?	13
6.1	Capabilities	13
6.2	Specific Examples	14
6.3	Biases and Toxicity	14
6.4	Emergent Abilities	15
7	Open Questions and Future Directions	15
7.1	Understanding Emergent Abilities	15
7.2	Practical Considerations for Research	16
7.3	Ethical Considerations	17
8	Summary	17
8.1	Key Takeaways	17
8.2	The Pre-training Paradigm	18
8.3	Looking Forward	18
9	Further Reading	18

1 Parameter-Efficient Fine-tuning

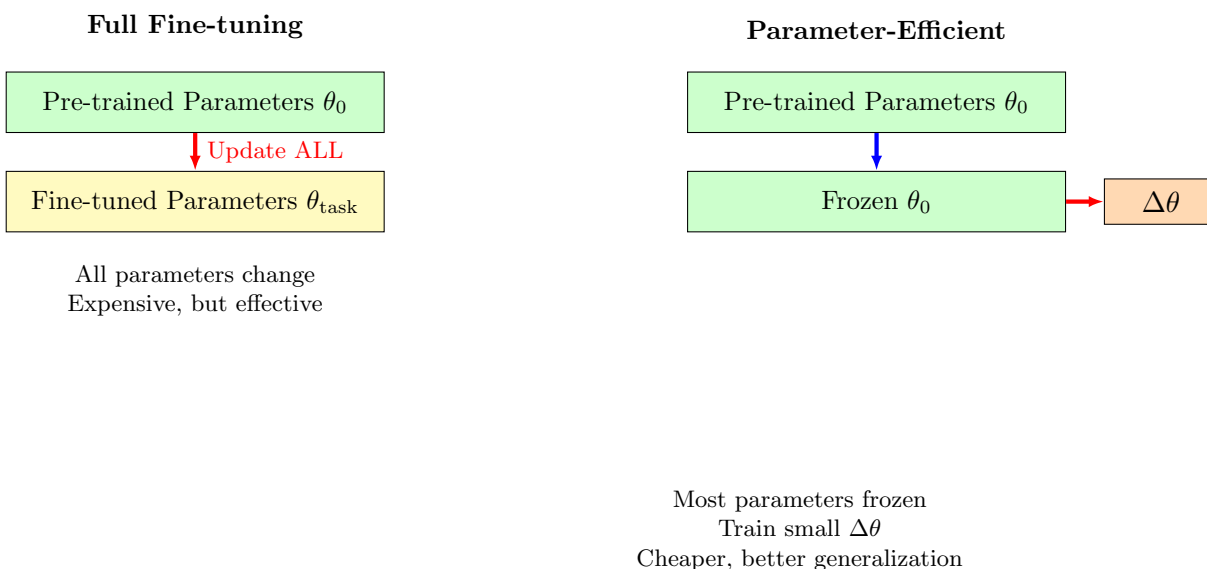
1.1 Motivation

Traditional fine-tuning updates **all parameters** of a pre-trained model. For large models (billions of parameters), this is:

- Computationally expensive (need to store gradients for all parameters)
- Memory intensive (optimizer states for all parameters)
- May overfit on small datasets
- Loses some generality from pre-training

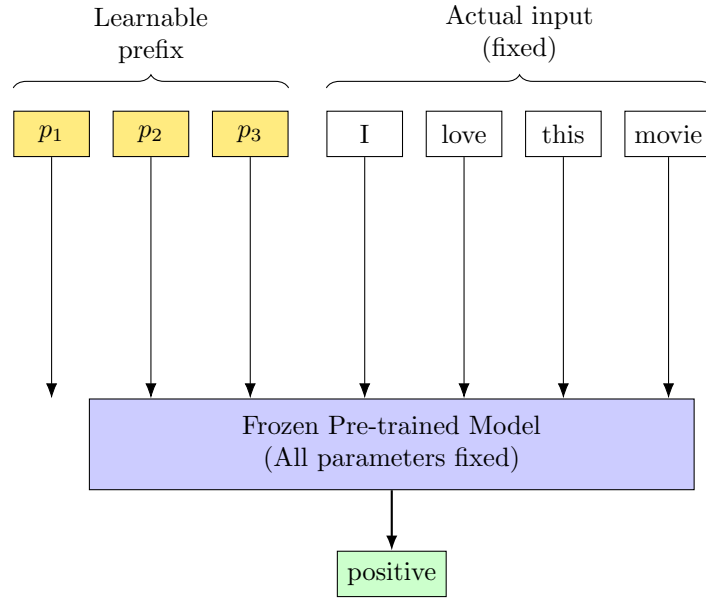
Goal: Make minimal changes to pre-trained parameters while adapting to new tasks.

1.2 Full Fine-tuning vs Parameter-Efficient Fine-tuning



1.3 Prefix Tuning / Prompt Tuning

Definition 1.1 (Prefix Tuning). Instead of modifying model parameters, prepend learnable “virtual tokens” to the input. Only these prefix embeddings are trainable; all model parameters remain frozen.



Training: Only update prefix embeddings p_1, p_2, p_3
 Model parameters never change!

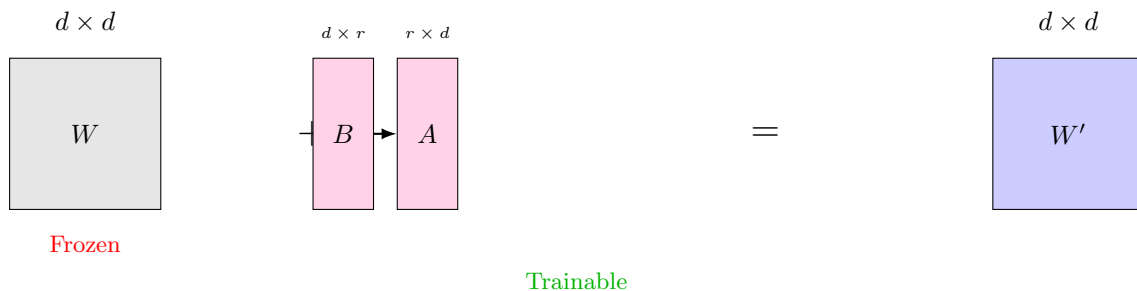
Key Properties:

- Prefix tokens are “fake” - not real words, just trainable vectors
- Typically 10-100 prefix tokens
- For decoders, must be at the beginning (can't see future context)
- Much cheaper: only train $\approx 0.1\%$ of parameters
- Preserves pre-trained knowledge better

1.4 LoRA: Low-Rank Adaptation

Definition 1.2 (LoRA). For each weight matrix $W \in \mathbb{R}^{d \times d}$ in the model:

- Freeze W
- Add trainable low-rank decomposition: $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$
- Effective weight: $W' = W + \Delta W = W + BA$
- Rank $r \ll d$ (typically $r = 4, 8, 16$)



Example:
 $d = 1024, r = 8$
 Original: $1024^2 = 1M$ params
 LoRA: $2 \times 1024 \times 8 = 16K$ params
60× reduction!

Why Low-Rank?

- Hypothesis: Task-specific adaptations live in low-dimensional subspace
- Dramatically reduces number of trainable parameters
- $\text{Params}_{\text{LoRA}} = 2dr$ vs $\text{Params}_{\text{full}} = d^2$
- Can apply to attention matrices (W_Q, W_K, W_V) and feed-forward layers

1.5 Comparison of Fine-tuning Methods

Method	Trainable Params	Memory	Performance
Full Fine-tuning	100%	High	Best
Prefix Tuning	$\approx 0.1\%$	Very Low	Good
LoRA	$\approx 0.1\text{-}1\%$	Very Low	Very Good
Train Last Layers	$\approx 10\%$	Medium	Moderate

Table 1: Comparison of fine-tuning approaches

2 Encoder-Decoder Pre-training

2.1 Challenge: Combining Bidirectional and Generative

Encoder-decoders have unique properties:

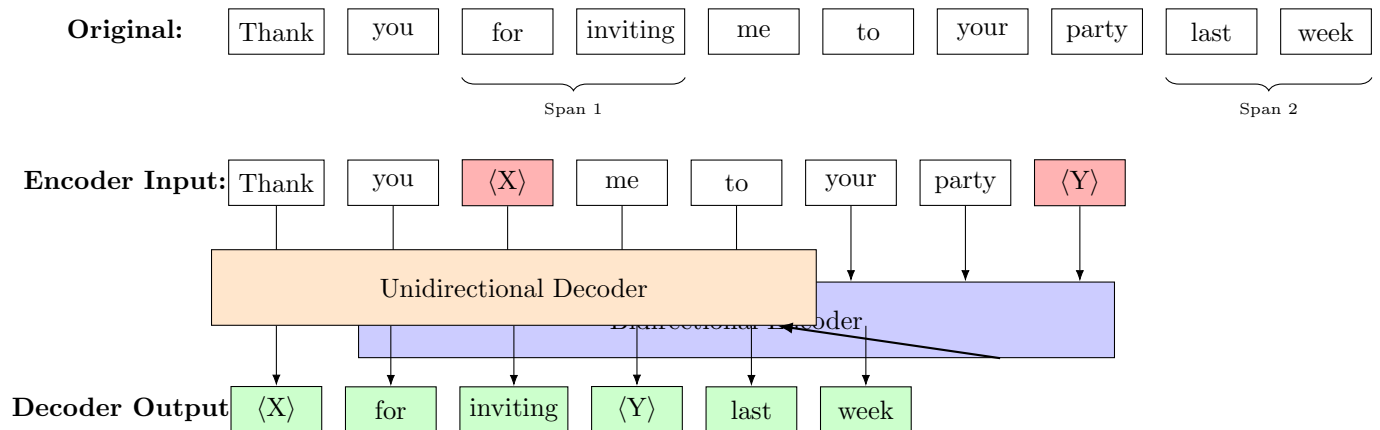
- Encoder: bidirectional context (good for understanding)
- Decoder: unidirectional generation (good for producing output)

Question: How to pre-train to leverage both?

2.2 Span Corruption (T5)

Definition 2.1 (Span Corruption). A pre-training objective that:

1. Corrupts the input by masking out contiguous spans of tokens
2. Encoder processes the corrupted input with bidirectional attention
3. Decoder generates the masked spans in order



Key Differences from BERT:

- BERT: Predict individual masked tokens in place
- T5: Generate all masked spans sequentially in decoder
- T5: Can handle variable-length spans naturally
- T5: More similar to generation tasks (machine translation, summarization)

2.3 T5: Text-to-Text Transfer Transformer

T5 frames **every NLP task** as text-to-text:

Translation: translate English to German: That is good. → Das ist gut.

All tasks as text-to-text

Question Answering: question: What is the capital of France? → Paris

Summarization: summarize: [long article] → [short summary]

Sentiment: sentiment: I love this movie! → positive

Advantages:

- Unified framework for all tasks
- Can leverage span corruption pre-training for generation
- Easy to add new tasks without architecture changes
- Same model, different text prompts

2.4 Implicit Knowledge Retrieval

Example 2.1 (Open-Domain Question Answering). T5 trained with span corruption can answer factual questions:

Pre-training: “Franklin D. Roosevelt was born in _____” $\rightarrow$ “1882”

Fine-tuning: Examples of Q&A pairs

Test: “When was Franklin D. Roosevelt born?” $\rightarrow$ “1882”

The model implicitly retrieves facts seen during pre-training!

Important caveat: Answers are often fluent but **frequently wrong**. The model cannot distinguish what it knows from plausible-sounding fabrications (“hallucinations”).

3 Decoder-Only Models: The GPT Family

3.1 Why Decoder-Only Models?

Advantages:

- Simpler architecture than encoder-decoder
- All parameters in one unified network
- Natural for text generation
- Scale efficiently to very large sizes

Pre-training: Standard left-to-right language modeling

$$\mathcal{L} = - \sum_{t=1}^T \log P(x_t \mid x_1, \dots, x_{t-1}; \theta) \quad (1)$$

3.2 GPT-1 (2018)

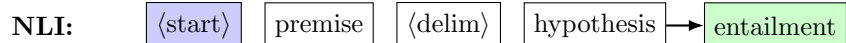
GPT-1 (Generative Pre-trained Transformer)

- Parameters: 117 million
- Layers: 12
- Hidden size: 768
- Attention heads: 12
- Vocabulary: 40K (BPE)
- Training data: BooksCorpus ($\approx$ 800M words)

First to show strong transfer learning
from language modeling pre-training

3.2.1 Fine-tuning for Tasks

GPT-1 can be fine-tuned for various tasks by formatting them as sequences:



Example:

The man is in the doorway ⟨delim⟩ The person is near the door
→ entailment

3.3 GPT-2 (2019): Scaling Up

GPT-2

- Parameters: **1.5 billion** ($13\times$ larger than GPT-1)
- Layers: 48
- Hidden size: 1600
- Training data: WebText ($\approx 40\text{GB}$, 9B words)

Key innovation: Much more data, larger scale

Capability: Coherent long-form text generation

Example 3.1 (GPT-2 Generation). **Prompt:** “In a shocking finding, scientists discovered a herd of unicorns living in a remote valley in the Andes Mountains.”

GPT-2 continuation: “Even more surprising to the researchers was the fact that the unicorns spoke perfect English. The scientist named the population, after their distinctive horn, Ovid’s Unicorn...” [continues coherently for several paragraphs]

This level of coherence was remarkable in 2019!

3.4 GPT-3 (2020): Emergence of In-Context Learning

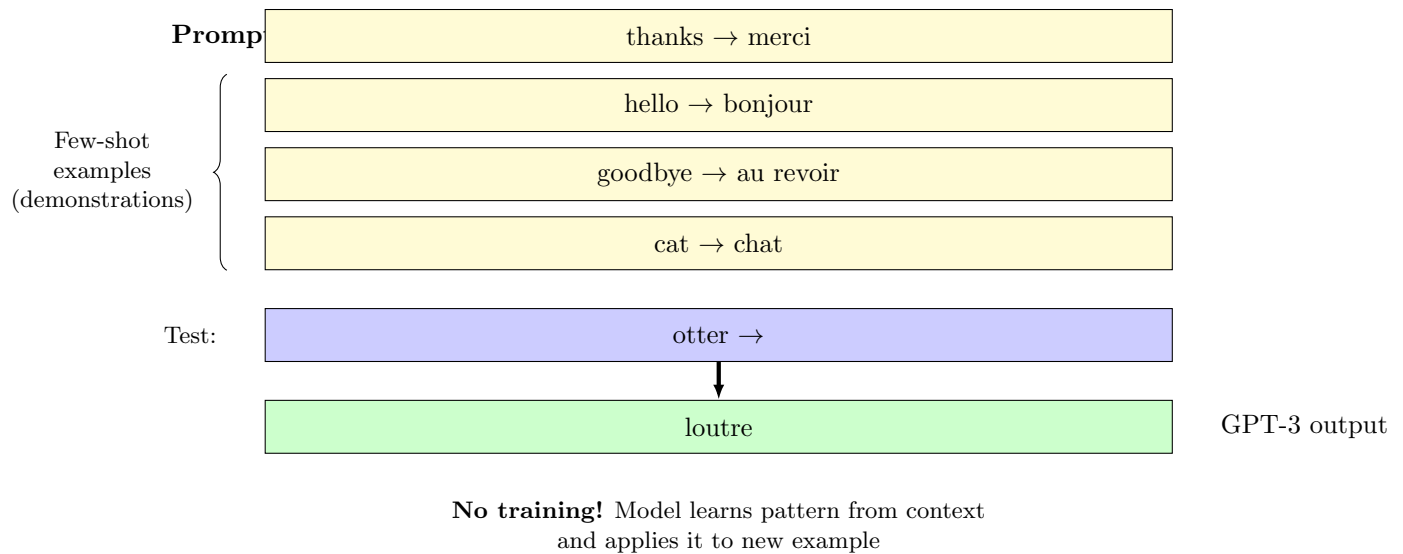
GPT-3

- Parameters: **175 billion** ($117\times$ larger than GPT-2)
- Layers: 96
- Hidden size: 12288
- Attention heads: 96
- Training data: 300B tokens (570GB)
- Training cost: **\$4-12 million**

Emergent ability: In-context learning without fine-tuning!

3.5 In-Context Learning

Definition 3.1 (In-Context Learning). The ability of a language model to perform tasks by conditioning on a few examples provided in the prompt, **without any gradient updates**.



3.5.1 Types of In-Context Learning

- **Zero-shot:** No examples, just task description

Translate to French: Hello →

- **Few-shot:** Multiple examples (typically 1-10)

English: Hello, French: Bonjour
 English: Goodbye, French: Au revoir
 English: Cat, French:

- **One-shot:** Single example

English: Hello, French: Bonjour
 English: Cat, French:

3.6 More In-Context Learning Examples

Example 3.2 (Arithmetic). **Prompt:**

Q: What is 2 + 3?
 A: 5
 Q: What is 7 + 12?
 A: 19
 Q: What is 25 + 36?
 A:

GPT-3: 61 ✓

Example 3.3 (Spelling Correction). **Prompt:**

Incorrect: I am hapy
 Correct: I am happy
 Incorrect: Thnak you
 Correct: Thank you
 Incorrect: I wnat to go
 Correct:

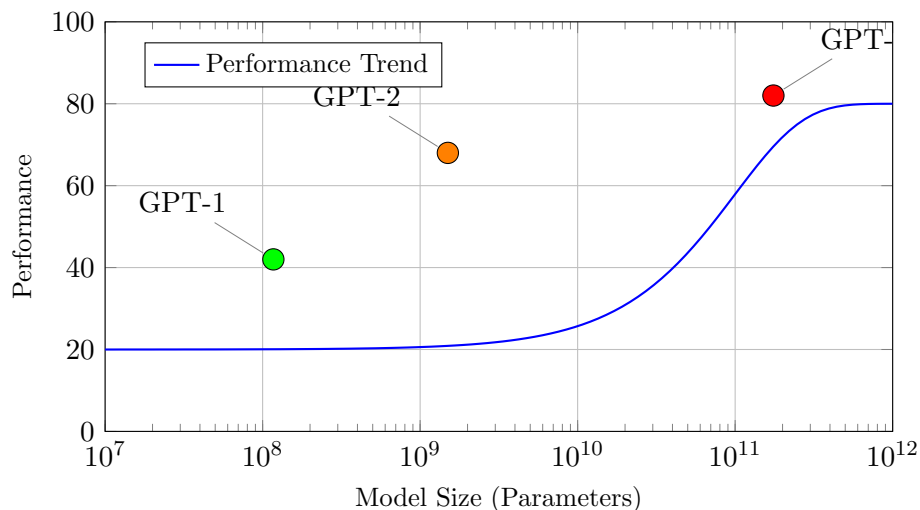
GPT-3: I want to go ✓

Surprising observation: The model identifies the pattern (task) from just the structure of the prompt!

4 Scaling Laws and Compute-Optimal Training

4.1 The Scaling Hypothesis

As models get larger, they consistently get better:



Key observation: Larger models consistently outperform smaller ones across diverse tasks.

4.2 The Compute Budget Problem

Training cost $\approx$ (Number of Parameters) $\times$ (Training Tokens)

Question: Given fixed compute budget, how should we allocate between:

- Making the model bigger?
- Training on more data?

4.3 Chinchilla Scaling Laws

Theorem 4.1 (Chinchilla Result (DeepMind, 2022)). GPT-3 was severely **overparameterized** and **undertrained**!

For compute-optimal training:

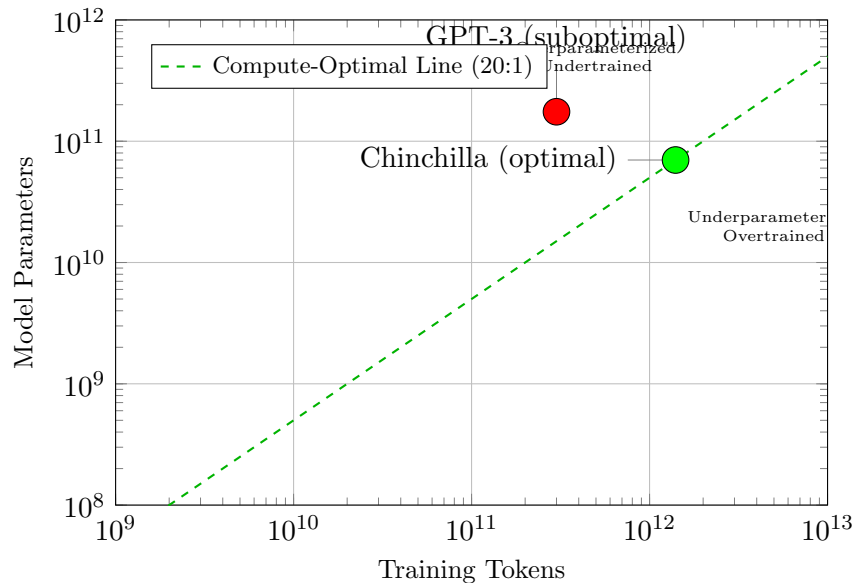
- Model size and training tokens should scale **proportionally**

- Rule of thumb: 20 tokens per parameter

GPT-3 Parameters: 175B Training tokens: 300B Ratio: 1.7 tokens/param → Overparameterized!	Too many parameters Not enough training
Chinchilla Parameters: 70B (40% of GPT-3) Training tokens: 1.4T (4.7× GPT-3) Ratio: 20 tokens/param → Better performance!	Balanced parameters and training data

Implication: With same compute budget, better to train a smaller model on more data than a huge model on less data.

4.4 Scaling Law Visualization



5 Chain-of-Thought Prompting

5.1 Motivation

Standard prompting often fails on complex reasoning tasks:

Standard Prompt:

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls.
Each can has 3 tennis balls. How many tennis balls does he have now?
A:

11 (Wrong!)

Model jumps directly to answer
without intermediate reasoning

5.2 Chain-of-Thought Prompting

Definition 5.1 (Chain-of-Thought (CoT)). Prompting technique where:

1. Demonstrations include explicit reasoning steps
2. Model learns to generate intermediate reasoning before final answer
3. “Let’s think step by step” pattern

Chain-of-Thought Prompt:

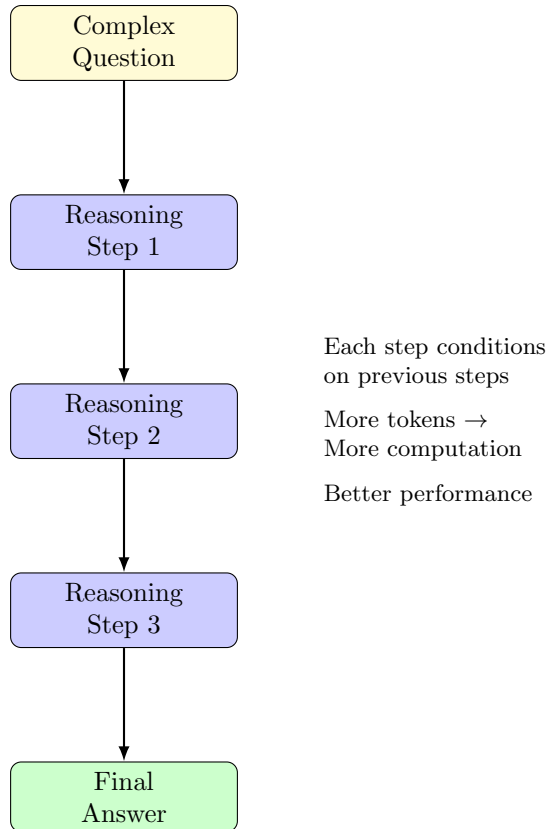
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls.
Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6
tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 for lunch and bought 6 more,
how many apples do they have?
A:

The cafeteria started with 23 apples. They used 20, so they have
 $23 - 20 = 3$ apples left. Then they bought 6 more, so $3 + 6 = 9$.
The answer is 9. ✓

5.3 Why Does Chain-of-Thought Work?

1. **Computational overhead:** Generating reasoning steps allows more computation time
2. **Decomposition:** Complex problem broken into simpler sub-problems
3. **Transparency:** Can inspect model’s reasoning process
4. **Error correction:** Model can “catch” its own mistakes mid-reasoning



5.4 Performance Gains

Chain-of-thought dramatically improves performance on reasoning tasks:

Task	Standard	Chain-of-Thought
Arithmetic (GSM8K)	18%	58%
Commonsense (StrategyQA)	54%	71%
Symbolic Reasoning	32%	65%

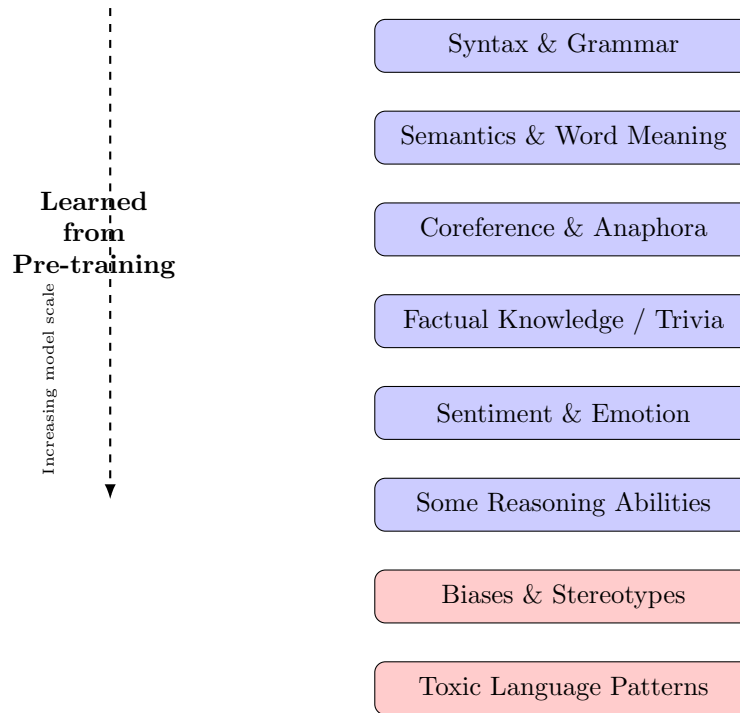
Table 2: Accuracy on reasoning benchmarks (GPT-3 175B)

Key insight: Emergence property of scale - only works well with very large models (>100B parameters).

6 What Do Pre-trained Models Learn?

6.1 Capabilities

Pre-trained language models acquire diverse capabilities:



6.2 Specific Examples

Example 6.1 (Syntax). Models learn grammatical structure:

- Subject-verb agreement
- Phrase structure
- Long-distance dependencies

Example 6.2 (World Knowledge). Models memorize facts from training data:

- “Paris is the capital of ___” → France
- “Steve Jobs founded ___” → Apple
- Geographic, historical, scientific facts

But: Cannot distinguish what it truly knows from plausible-sounding fabrications!

Example 6.3 (Reasoning). Some emergent reasoning abilities:

- Simple arithmetic (with chain-of-thought)
- Logical inference
- Multi-step problem decomposition

But: Fails on many reasoning tasks that require systematic thinking.

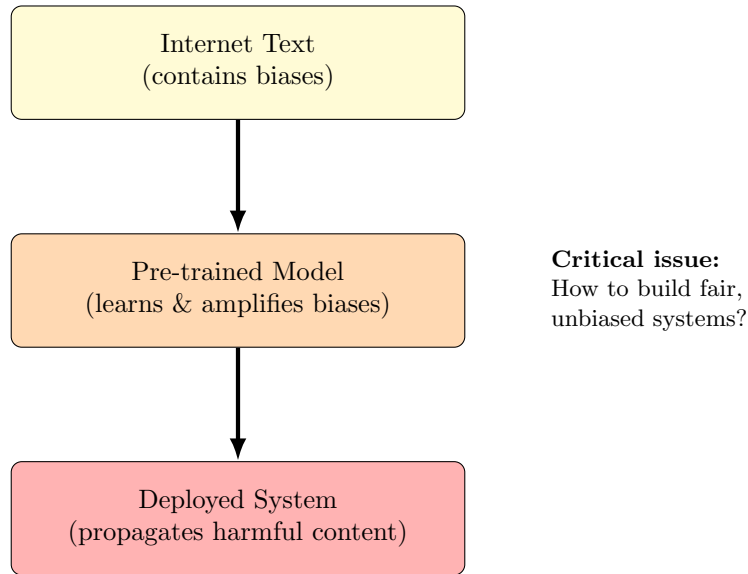
6.3 Biases and Toxicity

Problem 6.1 (Training Data Bias). Models learn from internet text, which contains:

- Gender stereotypes (“doctor” → “he”, “nurse” → “she”)
- Racial biases
- Religious and cultural prejudices

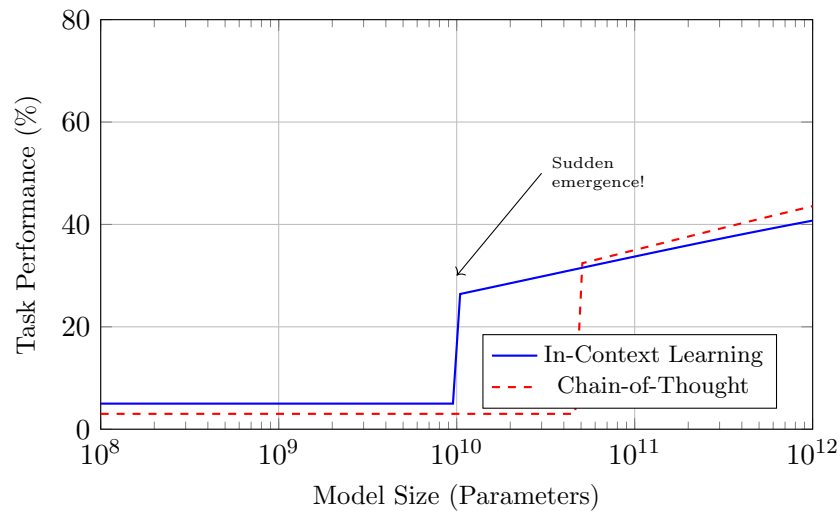
- Toxic and hateful language

These biases are **amplified** by the model, not just replicated.



6.4 Emergent Abilities

Some capabilities appear suddenly at scale:



Key question: What other abilities emerge at larger scales? We don't know until we build them!

7 Open Questions and Future Directions

7.1 Understanding Emergent Abilities

- Why do certain abilities emerge suddenly?

- Is it smooth improvement we can't measure?
- Or true phase transition in capabilities?
- **Can we predict what will emerge?**
 - What capabilities will 1T parameter models have?
 - Are there fundamental limits?
- **How do models perform reasoning?**
 - Is it pattern matching or true understanding?
 - Can we improve reasoning reliability?

7.2 Practical Considerations for Research

Using pre-trained models in your projects:

1. Model Selection

- Task: Classification/understanding? → BERT-style encoder
- Task: Generation? → GPT-style decoder
- Task: Seq2seq? → T5-style encoder-decoder

2. Fine-tuning Strategy

- Large dataset? → Full fine-tuning
- Small dataset? → Parameter-efficient (LoRA, prefix tuning)
- Domain-specific data? → Continued pre-training + fine-tuning

3. Prompting vs Fine-tuning

- No labeled data? → In-context learning (GPT-3+)
- Some labeled data? → Few-shot prompting or fine-tuning
- Lots of labeled data? → Fine-tuning

7.3 Ethical Considerations

Critical Challenges

Bias & Fairness: Models perpetuate societal biases

Toxicity: Can generate harmful content

Truthfulness: Confident but wrong (hallucinations)

Environmental Impact: Massive carbon footprint

Access: Only large organizations can train largest models

Responsibility: As practitioners, we must:

- Test for biases in our systems
- Implement safety measures
- Be transparent about limitations
- Consider societal impact

8 Summary

8.1 Key Takeaways

1. Parameter-Efficient Fine-tuning

- Prefix tuning: Train only input embeddings
- LoRA: Train low-rank weight updates
- Reduces cost, improves generalization

2. Encoder-Decoder Pre-training

- Span corruption (T5): Mask spans, generate them
- Text-to-text framework: Unified interface for all tasks
- Good for seq2seq tasks

3. Decoder-Only Models (GPT Family)

- GPT-1 (117M): Showed transfer learning works
- GPT-2 (1.5B): Coherent long-form generation
- GPT-3 (175B): In-context learning emerges

4. In-Context Learning

- Learn tasks from demonstrations in prompt

- No gradient updates needed
- Emergent ability at large scale

5. Scaling Laws

- Chinchilla: Balance parameters and training tokens
- GPT-3 was overparameterized and undertrained
- Compute-optimal: ≈ 20 tokens per parameter

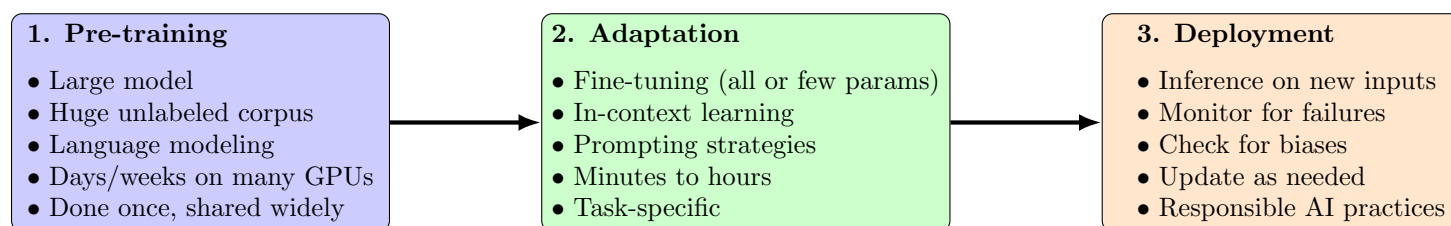
6. Chain-of-Thought

- Show reasoning steps in demonstrations
- Model learns to reason step-by-step
- Dramatically improves reasoning tasks

7. What Models Learn

- Syntax, semantics, facts, some reasoning
- Also: biases, toxicity, misinformation
- Emergent abilities at scale

8.2 The Pre-training Paradigm



8.3 Looking Forward

The field continues to evolve rapidly:

- Even larger models (GPT-4, PaLM, Claude, etc.)
- Better alignment with human values (RLHF)
- Multimodal models (text + images + audio)
- More efficient training and inference
- Better understanding of capabilities and limitations

Core insight: Pre-training on large-scale data with simple objectives (language modeling, span corruption) produces models with remarkable and sometimes surprising general capabilities.

9 Further Reading

- **GPT-1:** Radford et al. (2018). “Improving Language Understanding by Generative Pre-Training”
- **GPT-2:** Radford et al. (2019). “Language Models are Unsupervised Multitask Learners”
- **GPT-3:** Brown et al. (2020). “Language Models are Few-Shot Learners”

- **T5:** Raffel et al. (2020). “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”
- **Scaling Laws:** Hoffmann et al. (2022). “Training Compute-Optimal Large Language Models” (Chinchilla)
- **Chain-of-Thought:** Wei et al. (2022). “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”
- **LoRA:** Hu et al. (2021). “LoRA: Low-Rank Adaptation of Large Language Models”
- **Prefix Tuning:** Li & Liang (2021). “Prefix-Tuning: Optimizing Continuous Prompts for Generation”